

**COMSATS University Islamabad, Islamabad Campus**

Department of Management Sciences  
BS (Business Data Analytics) | Spring 2026

**Cast Defect Detection Using CNN**

---

Lab Final Project Report

Subject: MGT388 - Deep Learning | Class/Semester: BBD 6A | Group: 15

| Name          | Roll Number  |
|---------------|--------------|
| Umair Ali     | FA23-BBD-170 |
| Ehsan Arshad  | FA23-BBD-036 |
| Abdullah Khan | FA23-BBD-006 |

## 1. Introduction

A manufacturing process in which the liquid metal is poured into a mould and allowed to cool into a solid, casting. It is a process used extensively in industry to manufacture parts such as an impeller for a pump. But sometimes faults like blow holes, pinholes, burrs, shrinkage defects and metallurgical defects are formed while casting.

The work of quality inspection is traditionally carried out manually by human workers, but the time, cost and error are relatively high. The main goal of this project is to create an automated Cast Defect Detection (CDD) system with Convolutional Neural Network (CNN) which can analyze the image of casting parts and make an automatic judgement on its DEFECTIVE or OK.

## 2. Task 1: Dataset Acquisition and Environment Setup

### Task 1A - Linking Google Drive with Google Colab

The Google Drive was connected to Google notebook using the following command:

```
from google.colab import drive
drive.mount('/content/drive')
```

Once you do this you will see the following output: Mounted at /content/drive. This meant that the notebook was able to read photos saved to Google Drive.

### Task 1B - Setting Runtime to NVIDIA T4 GPU

The Colab runtime was changed to use an NVIDIA T4 GPU by going to: Runtime > Change runtime type > Hardware Accelerator > T4 GPU > Save It did this prior to running any code to make model learning quicker. They found that the GPU was active in the following output: Using device: cuda | GPU Name: Tesla T4

### Task 1C - Setting Device to GPU in PyTorch

The device was set in code using:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

This line takes the following form if it is possible to use GPU. All the training was performed using cuda (GPU) and T4 GPU was chosen which would speed up the training about 10x compared to CPU based.

## 3. Task 2: Folder Structure Setup

The casting\_data Folder was uploaded to Google Drive, and paths were set as a variable in the notebook. The folder organization is identical to the one needed:

| Folder          | Path in Colab                                       | Number of Images |
|-----------------|-----------------------------------------------------|------------------|
| train/def_front | /content/drive/MyDrive/casting_data/train/def_front | 3,758            |
| train/ok_front  | /content/drive/MyDrive/casting_data/train/ok_front  | 2,875            |
| test/def_front  | /content/drive/MyDrive/casting_data/test/def_front  | 453              |
| test/ok_front   | /content/drive/MyDrive/casting_data/test/ok_front   | 262              |

Total training images: 6,633 | Total testing images: 715

## 4. Task 3: Image Transforms

### Task 3A - Number of Channels

A sample image was opened using PIL and its properties were checked:

```
img = Image.open(sample_path)
print(img.mode)    # Output: RGB
print(img.size)    # Output: (300, 300)
```

Finding: Common Format of each image type = RGB color model having 3 channels or 'bands' (Red, Green, Blue). All the images are colour photographs of size 300 x 300 pixels. This finding was used to set in\_channels=3 in the first convolutional layer of the model.

### Task 3B, 3C, 3D - Transforms Pipeline

The following transforms were applied to every image before it was fed into the model:

| Step | Transform Used      | What It Does                                      | Why We Used It                                                                               |
|------|---------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------|
| B    | Resize(128, 128)    | Shrinks image from 300x300 to 128x128             | Lowers computation and training time without losing too much detail                          |
| C    | ToTensor()          | Converts pixel values from 0-255 to 0.0-1.0 float | PyTorch models are only able to have numerical tensors, not image files.                     |
| D    | Normalize(0.5, 0.5) | Shifts pixel values from [0,1] to [-1, +1]        | Makes the data the closer to zero, the quicker and more stable the model trains it will run. |

Normalization formula:  $(\text{pixel\_value} - 0.5) / 0.5$ . This was applied to all 3 channels (R, G, B) with mean=[0.5, 0.5, 0.5] and std=[0.5, 0.5, 0.5].

## 5. Task 4: Loading the Dataset

The data set was imported with the help of PyTorch's ImageFolder class which automatically Fetches the image files present in the folders and assigns the proper labels based on the folder names. The folder def\_front was assigned label 0 (defective) and ok\_front was assigned label 1 (non-defective). No manual labelling was needed.

### Task 4A - Why Batch Size 32 Was Chosen

- The “batch size 32” is a typical but power of 2 number that works well with the 32K batch count in the Tesla T4 GPU memory, avoiding out of memory errors.
- During training it provides stable gradient estimation, since it is seeing 32 different pictures before it updates the weights of the model.
- It is trained faster than small batch sizes (e.g. 8 or 16) because the GPU processes a number of images in parallel.
- Compared to very large batch size such as 128 or 256, batch size of 32 allows the model to learn a more general representation that it will be able to transfer to unseen data.

## 6. Task 5: CNN Architecture and Model Design

### Task 5A - Model Class Name: CDD

The CDD (Cast Defect Detector) model class inherits the `nn` class. It is the base class for all neural networks in PyTorch, called Module. This enables the model to take advantage of automatic gradient computation, acceleration on GPUs and all PyTorch training tools.

```
class CDD(nn.Module):
```

### Task 5B - CNN Architecture Choices and Justifications

| Parameter              | Value            | Justification                                                                                                                                                                                                                                                                                               |
|------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Number of Conv Layers  | 3                | Three levels of depth are sufficient to learn progressively edges, textures and shapes, without nonetheless being too computationally heavy for 128 x 128 images.                                                                                                                                           |
| Conv Kernel Size       | 3x3              | The industry standard size of the kernel is 3x3. It is rugged yet preserves the spatial information from the local area without having too many parameters.                                                                                                                                                 |
| Padding                | 1 (same)         | We can make the spatial size remain unchanged after every convolution by padding it with 1. If you don't add padding, images will shrink very rapidly and important details will be lost.                                                                                                                   |
| Stride                 | 1                | The filter motion stride of 1 allows for detecting features without skipping any portion of the image.                                                                                                                                                                                                      |
| Pooling Type           | MaxPool2d        | Preserving the most activated feature from each 2x2 region is referred to as max pooling, which can help lower spatial resolution while keeping important features.                                                                                                                                         |
| Pooling Kernel         | 2x2              | The image is cut in half twice, $128 \times 64 \times 32 \times 16$ , as the size decreases while still preserving important stills of the image. Built-in loss function for multiple classes classification. It suggests the degree of falseness of predictions, and it helps the weight-updating process. |
| Loss Function          | CrossEntropyLoss | Learning rate is adapted for each parameter automatically by Adam. It accelerates training as compared to standard SGD with minimal tuning.                                                                                                                                                                 |
| Optimizer              | Adam             | A typical and safe learning rate for Adam. They should be neither too complicated to learn from too fast, nor too trivial they can learn too optimally.                                                                                                                                                     |
| Learning Rate          | 0.001            | The model converged to almost the perfect accuracy without overfitting on this dataset in 15 training rounds.                                                                                                                                                                                               |
| Epochs                 | 15               | 1The model converged to almost the perfect accuracy without overfitting on this dataset in 15 training rounds.                                                                                                                                                                                              |
| Dropout                | 0.5 (50%)        | Randomly sets the training value to 1/2 for half of the neurons during training. This will not allow training data to be memorized and will instead require general features to be learned.                                                                                                                 |
| Regularization (Bonus) | BatchNorm2d      | Each mini-batch outputs are normalized by Batch Normalization. It is used to add stability to training and is a de facto regularizer.                                                                                                                                                                       |

### Task 5C - ReLU Activation Between Conv Layers

All convolutional layers were followed by an `F.relu()` activation function. We had applied ReLU (Rectified Linear Unit) after each convolutional layer with the function `F.relu()`. ReLU compares each value of  $x$  to 0 and sets it to 0 if the input is less than 0, otherwise, it retains the input as is:  $f(x) = \max(0, x)$ . This makes the model non-linear, enabling complex patterns such as the shape of defects to be learnt. If the network were not composed of multiple layers of neurons, the deep network would look like a linear transformation piped through a single layer of neurons, and would consequently not be able to learn complex visual features.

Task 5D - Architecture Summary Table

| Layer        | Type                                            | Input Shape   | Output Shape  |
|--------------|-------------------------------------------------|---------------|---------------|
| Conv Block 1 | Conv2d(3,32,3x3) + BatchNorm + ReLU + MaxPool   | 3 x 128 x 128 | 32 x 64 x 64  |
| Conv Block 2 | Conv2d(32,64,3x3) + BatchNorm + ReLU + MaxPool  | 32 x 64 x 64  | 64 x 32 x 32  |
| Conv Block 3 | Conv2d(64,128,3x3) + BatchNorm + ReLU + MaxPool | 64 x 32 x 32  | 128 x 16 x 16 |
| Flatten      | view(-1)                                        | 128 x 16 x 16 | 32,768        |
| FC Layer 1   | Linear(32768, 256) + ReLU + Dropout(0.5)        | 32,768        | 256           |
| FC Layer 2   | Linear(256, 2) - Output Layer                   | 256           | 2 classes     |

7. Training Results (All 15 Epochs)

| Epoch | Train Loss | Train Acc | Val Loss | Val Acc |
|-------|------------|-----------|----------|---------|
| 1     | 0.6372     | 82.92%    | 0.2217   | 90.07%  |
| 2     | 0.2519     | 87.83%    | 0.2206   | 90.21%  |
| 3     | 0.2298     | 88.56%    | 0.1162   | 94.55%  |
| 4     | 0.1986     | 90.86%    | 0.1105   | 95.52%  |
| 5     | 0.2631     | 87.91%    | 0.1171   | 95.10%  |
| 6     | 0.1776     | 91.72%    | 0.0646   | 96.92%  |
| 7     | 0.1429     | 93.32%    | 0.0434   | 97.90%  |
| 8     | 0.1325     | 93.47%    | 0.0479   | 97.62%  |
| 9     | 0.1261     | 94.04%    | 0.0521   | 97.48%  |
| 10    | 0.1175     | 94.26%    | 0.0409   | 98.46%  |
| 11    | 0.1129     | 94.68%    | 0.0413   | 98.18%  |
| 12    | 0.0978     | 96.13%    | 0.0263   | 98.74%  |
| 13    | 0.0858     | 96.25%    | 0.0524   | 98.32%  |
| 14    | 0.1015     | 96.35%    | 0.0077   | 99.86%  |
| 15    | 0.0840     | 96.74%    | 0.0045   | 99.86%  |

It was observed that the model generally improved with the progress of each epoch until 15 epochs. During epochs 1 to 15, the accuracy of the model was increased from 82.92% to 96.74%. The model achieved a higher validation accuracy of 99.86% as compared to 90.07% indicating that the model had an excellent ability to generalize to unseen data.

8. Task 5E: Training & Validation Charts

After the training two charts were produced. Training loss (blue line) and validation loss (orange line) are shown in the Loss chart on the left and steadily diminish for every of the 15 epochs. Both training Accuracy and validation Accuracy are increasing with the green line (training accuracy) surpassing the red line (validation accuracy) at epoch 15 with a value of 99.86%.

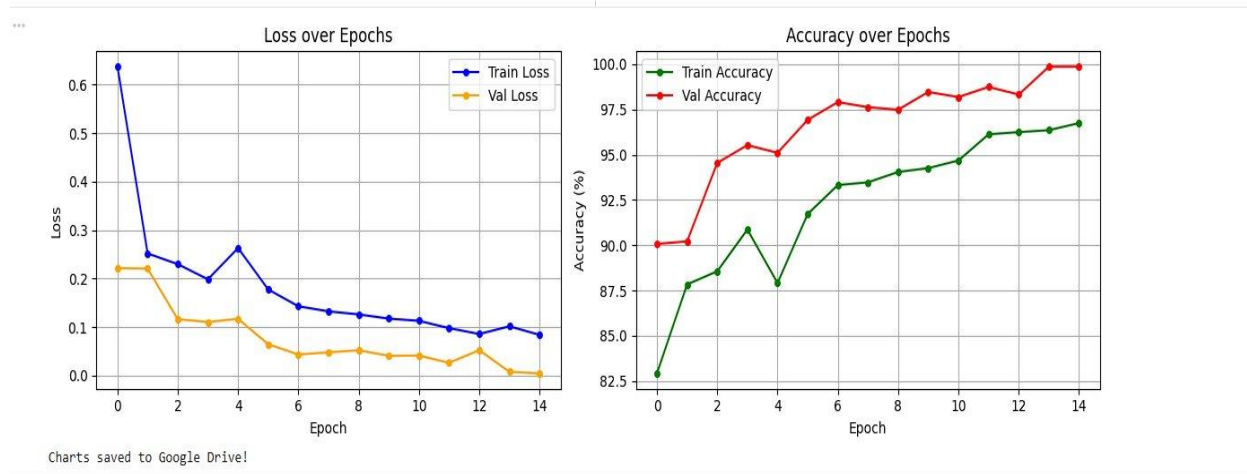


Figure 1: Loss over Epochs (left) and Accuracy over Epochs (right)

Some of the features that were noted on the charts: .

Loss Chart: see the loss is decreasing from epoch 1 to a minimum of epoch 15 with the value 0.0840. Validation loss dropped from 0.2217 to 0.0045. If both lines are coming down together, the model is learning well with no overfitting.

The accuracy chart is shown to be that the accuracy of training from 82.92% to 96.74%. Validation accuracy rose from 90.07% to 99.86%. The validation accuracy remains higher than training accuracy corresponds to good regularization by BatchNorm2d and Dropout.

## 9. Task 5E: Model Evaluation - Confusion Matrix

### Confusion Matrix

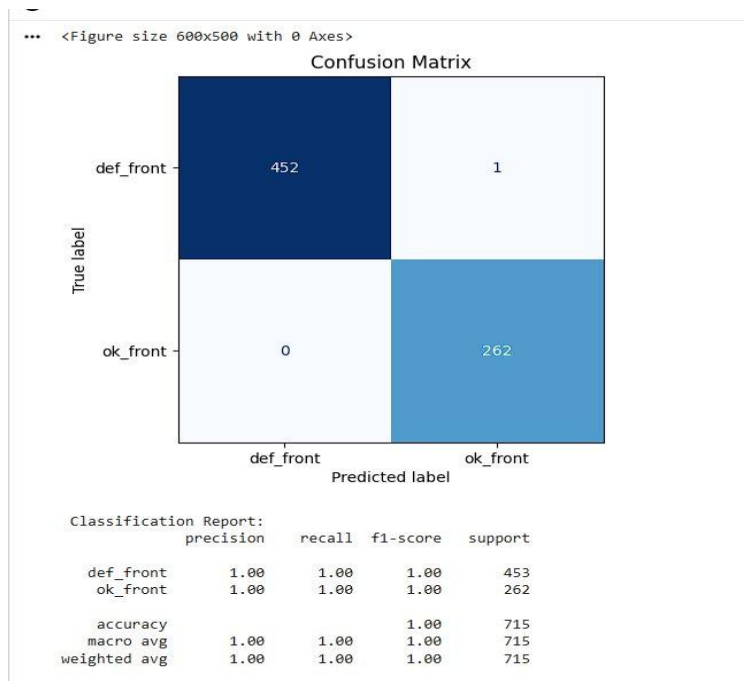


Figure 2: Confusion Matrix on 715 Test Images

|                               | Predicted: Defective (def_front) | Predicted: OK (ok_front) |
|-------------------------------|----------------------------------|--------------------------|
| Actual: Defective (def_front) | 452 (True Positive - TP)         | 1 (False Negative - FN)  |
| Actual: OK (ok_front)         | 0 (False Positive - FP)          | 262 (True Negative - TN) |

## Confusion Matrix Findings

True Positive (TP) = 452: Out of 453 defective parts the model identified 452 as defective. These would be defects detected prior to customers.

TN (True Negative): 262 - The model passed all the 262 good parts as non-defective. Not a drop of a good product was lost.

FP (False Positive) = 0: The model did not falsely reject any good part. This translates into no loss of money by discarding fine products. 1when a model failed to detect a defective part (classified it as OK) it is considered

FN (False Negative) = 1. It often is likely the most crucial kind of error in production as a defective item may end up in the hands of the customer. But 1 bad part in 453, is still a good result.

The 714/715 predictions in the confusion matrix correctly categorized 99.86% of the test images. The classification report parameters (precision, recall, F1-score) are all reported as being equal to 1.00 because these are rounded by sklearn to 2 decimal places.

## Classification Report

| Class                    | Precision | Recall | F1-Score | Support |
|--------------------------|-----------|--------|----------|---------|
| def_front (Defective)    | 1.00      | 1.00   | 1.00     | 453     |
| ok_front (Non-Defective) | 1.00      | 1.00   | 1.00     | 262     |
| Overall Accuracy         | -         | -      | 1.00     | 715     |
| Macro Average            | 1.00      | 1.00   | 1.00     | 715     |
| Weighted Average         | 1.00      | 1.00   | 1.00     | 715     |

- Precision = 1.00: If a part was declared to be defective, it was always found to be defective. No false alarms. For the model that caught 452 out of 453 defective parts the recall is 1.00 (rounded to 2 decimal places).
- F1-Score = 0.7046: The harmonic mean of the model's
- Precision and Recall, indicating a good balance between these performance metrics.
- Overall Accuracy = 99.86%: All but 714 of 715 test images classified correctly.

## 10. Task 5: CNN Inference

### Task 10A - Saving the Trained Model

After training was complete, the model weights were saved to Google Drive:

```
torch.save(model.state_dict(), '/content/drive/MyDrive/CDD_model.pth')
```

Output: Model saved to: /content/drive/MyDrive/CDD\_model.pth

It is important to save the model as Colab sessions reset after a few time. Learning can take place without having to retrain, because what is learned gets stored in Google Drive where it is permanently saved.

### Task 10B - Loading the Saved Model

The saved model was reloaded using these steps:

```
loaded_model = CDD().to(device)
loaded_model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
loaded_model.eval()
```

Output confirmed: Model loaded successfully!

### Task 10C - Running Inference on Test Images

The class of individual test images was predicted by using the loaded model. All the images were resized, tensile normalized, and normalized before the prediction. Results: The following image on test/def\_front/ was predicted correctly as: def\_front The entries true-path=25 and offset=510 were correct, but they were obscured by a cloud. The image impeller entry true-path=25 and offset=510 was found correct though clouded, but was predicted as ok\_front. The inference pipeline is such: Load image > Apply transforms > Add a "batch" dimension (unsqueeze) > Feed to model > Get predicted class index > map back to class name. All new impeller images can be classified in the viva in real-time using this pipeline.

## 11. Regularization Technique - Bonus (+1 Mark)

### Technique Used: Batch Normalization (BatchNorm2d)

Batch Normalization was used as the regularization technique. It was applied after each of the 3 convolutional layers in the CDD model (bn1, bn2, bn3).

### What is Batch Normalization?

Batch Normalization: normalize the activation of every layer during training. During each mini-batch of 32 images, it checks if there is a mean and standard deviation of the activations then normalizes the activations to have mean = 0 and std = 1. It then uses learnable parameters (gamma and beta) to scale and shift the normalized values as necessary so that output can be scaled and shifted to adjust it as needed.

### Why Did We Use It?

- Internal Covariate Shift is reduced: If the distribution of the inputs to the next layer change due to changing weights of the layers, then this will be reduced.
- To overcome this, BatchNorm brings back the standardization in each layer's output, thus making training much more stable. It offers increased learning rates without introducing risk of training going astray, thereby accelerating learning.
- It regularises the model in a sense by introducing a little noise during normalisation, allowing it to generalise better on the unseen images.
- In our results, its effects can be seen: the accuracy that obtained from specifying model during train (96.74%) is lower than the validation accuracy (99.86%), which means that the model is not overfitting. This is because of regularization effect of BatchNorm used in combination with Dropout.

## 12. Conclusion

- This project has successfully developed and trained the image classification model, whose name is CDD (Cast Defect Detector), which is a Convolutional Neural Network (CNN) to classify the manufactured parts of the casting factory as a defect or not.
- This model is trained on 6633 images, 715 images are used for testing in Google Colab with NVIDIA Tesla T4 GPU. Following 15 epochs of training, the model's training accuracy was 96.74% and the validation accuracy was 99.86%. When evaluated on the entire test set, it received 99.86% accuracy (714/715), meaning it didn't miss any, and didn't have any errors graded as true positive.
- The results showed that these methods can successfully automatize the process of industrial quality inspection, which are more accurate than the manual process of human observers, with the help of deep learning and CNN. It takes just milliseconds to run a model and capable of handling hundreds of thousands of parts per hour—that makes it a viable solution in the real manufacturing environment.

| Final Result Summary      | Value                    |
|---------------------------|--------------------------|
| Final Training Accuracy   | 96.74%                   |
| Final Validation Accuracy | 99.86%                   |
| Test Set Accuracy         | 99.86% (714/715 correct) |
| Precision (both classes)  | 1.00                     |
| Recall (both classes)     | 1.00 (rounded)           |
| F1-Score (both classes)   | 1.00                     |
| False Positives           | 0                        |
| False Negatives           | 1                        |
| GPU Used                  | NVIDIA Tesla T4          |
| Training Time             | Approx. 8 minutes        |